Almacenamiento de información.

Caso práctico

Antes de irse de vacaciones con su familia, **María y Félix tienen una reunión con el responsable del departamento de informática de la empresa, Juan**, en la que éste último les explica que aún se puede mejorar la gestión de datos de la empresa.

Juan opina que ya que en la empresa **se ha impuesto el uso de la tecnología XML**, sería mejor utilizar un sistema de almacenamiento y consulta de datos compatible con dicha tecnología.

Van a intentar utilizar un sistema de almacenamiento de datos, diferente de la base de datos relacional que han usado hasta ahora, que es compatible con la tecnología XML que están utilizando.

Además, para acceder a los datos de esta base de datos nativa van a utilizar un lenguaje de consultas llamado **XQuery**, que equivale al SQL usado con las bases de datos relacionales.

1.- Utilización de XML para almacenamiento de información.

Como ya hemos visto, XML, es un estándar potente y de amplia aceptación para guardar y comunicar información acerca de objetos. Permite la codificación de información separada de la forma en la que se debe presentar al usuario. Para encontrar un fragmento específico de información en los contenidos de un nodo o atributo XML, habrá que procesar completamente el archivo XML.

Una base de datos XML se puede ver como una colección de documentos XML, en la que cada documento representa un registro.

Cada documento XML es un archivo en el sistema de archivos y contiene una cadena válida XML.

La estructura de un documento XML suele seguir un mismo esquema XML, aunque no es necesario que sea así. Este es uno de los beneficios de las bases de datos XML, ya que cada archivo se puede configurar de forma estructurada por lo que es independiente, pero fácilmente accesible.

El tener colecciones de documentos con un esquema independiente proporciona a la base de datos flexibilidad, y facilita el desarrollo de la aplicación.

En los últimos años, se ha visto necesaria la existencia de estándares de intercambio de información, con el objetivo de que las organizaciones puedan compartir su información de una manera más cómoda, automática y eficiente.

Autoevaluación

La tecnología XML:

☐ Permite tratar a los ficheros como si fuesen base de datos.
☐ Facilita el desarrollo de las aplicaciones al tener el esquema independiente de los datos.
☐ Hace que el compartir información sea más cómodo.
☐ Permite el acceso directo a los datos buscados, es decir sin recorrer todo el archivo.

Mostrar Información

1.1.- Ámbitos de aplicación.

Los documentos y los requerimientos de almacenamiento de datos XML pueden ser agrupados en dos categorías generales:

- **Centrados en los datos.**
  Suelen incluir documentos de facturación típicos, órdenes de compra, y documentos menos estructurados. Es apropiada para ítems como contenidos de periódicos, artículos y publicidad.

  Si el documento XML tiene una estructura bien definida y contiene datos que pueden ser actualizados y usados de diversos modos, el documento es habitualmente centrado en los datos.
- **Centrados en los documentos.**
  Tienden a ser más impredecibles en tamaño y contenido que los centrados en los datos, los cuales son altamente estructurados, con tipos de datos de tamaño limitado y reglas menos flexibles para campos opcionales y contenido.

  Los sistemas de almacenamiento XML deben acomodarse eficientemente con ambos tipos de requerimientos de datos, dado que XML está siendo usado en sistemas que administran ambos tipos de datos. La mayoría de los productos se enfocan en servir uno de esos formatos de datos mejor que el otro.

  Las bases de datos relacionales tradicionales son mejores para tratar con requerimientos centrados en los datos, mientras que los sistemas de administración de contenido y de documentos, suelen ser mejores para almacenar datos centrados en el documento.

  Los sistemas de bases de datos deben ser capaces de exponer los datos relacionales como XML, y almacenar el XML recibido como datos relacionales para transferir, obtener y almacenar los datos requeridos por la aplicación.

Autoevaluación

Elige las características que se ajusten a los sistemas centrados en datos:

☐ Contiene datos con una estructura no muy definida.
☐ Es apropiada para publicidad.
☐ Los datos pueden actualizarse.
☐ Contiene datos con una estructura muy definida.

Mostrar información.

2.- Sistemas de almacenamiento de información.

Una expresión XPath es un predicado que se aplica sobre una estructura de árbol correspondiente a la jerarquía de los datos de un documento XML y devuelve todo lo que encaja con ese predicado.

En lo que concierne a las bases de datos, XML permite integrar sistemas de información hasta ahora separados:

- **Sistemas de información basados en documentos** (ficheros), tienen estructura irregular, utilizan tipos de datos relativamente simples y dan gran importancia al orden.
- **Sistemas de información estructurados** (bases de datos relacionales), son relativamente planos, utilizan tipos de datos relativamente complejos y dan poca importancia al orden.

Podemos establecer las siguientes semejanzas entre una base de datos y un fichero XML con su esquema asociado:

- La tecnología XML usa uno o más documentos para almacenar la información.
- Define esquemas sobre la información.
- Tiene lenguajes de consulta específicos para recuperar la información requerida.
- Dispone de APIs (SAX, DOM).

Pero aparecen muchas más cosas que lo diferencian. Debido a que no es una base de datos, la tecnología XML carece, entre otras cosas, tanto de almacenamiento y actualización eficientes como de índices, seguridad, transacciones, integridad de datos, acceso concurrente, disparadores, etc.; que son algunas de las características habituales en las bases de datos. Por tanto es imposible pensar que XML se vaya a utilizar para las tareas transaccionales de una organización para las cuales sigue estando sobradamente más justificado utilizar una base de datos.

Autoevaluación

Marcar las afirmaciones que son correctas sobre bases de datos XML:

☐ Integran sistemas de información basados en documentos.
☐ Integran sistemas de información estructurados.
☐ La información se estructura siguiendo esquemas definidos previamente.
☐ Garantiza la actualización eficiente.

Mostrar Información

2.1.- XML y BD relacionales.

Las **bases de datos relacionales** se basan en las relaciones (tablas bidimensionales), como único medio para representar los datos del mundo real. Están asociadas al lenguaje estándar SQL.

Se han creado complejas teorías y patrones para encajar objetos o estructuras jerarquizadas en bases de datos relacionales.

Existen numerosos middlewares encargados de la transferencia de información entre estructuras XML y bases de datos relacionales.

Las bases de datos relacionales suponen una posibilidad para el almacenamiento de datos XML. Sin embargo, no están bien preparadas para almacenar estructuras de tipo jerárquico como son los documentos XML, algunas de las causas son:

- Las bases de datos relacionales tienen una estructura regular frente al carácter heterogéneo de los documentos XML.
- Los documentos XML suelen contener muchos niveles de anidamiento mientras que los datos relacionales son planos.
- Los documentos XML tienen un orden intrínseco mientras que los datos relacionales son no ordenados.
- Los datos relacionales son generalmente densos (cada columna tiene un valor), mientras que los datos XML son dispersos, es decir, pueden representar la carencia de información mediante la ausencia del elemento.

Algunas de las razones para usar los tipos de bases de datos relacionales y los productos de bases de datos existentes para almacenar XML, aún cuando no sea de forma nativa son:

- Las bases de datos relacionales y orientadas a objetos son bien conocidas, mientras que las bases de datos XML nativas son nuevas.
- Como resultado de la familiaridad con las bases de datos relacionales y orientadas a objetos, los usuarios se inclinan a ellas especialmente por el rendimiento.

Autoevaluación

El hecho de que el uso de las bases de datos relacionales esté muy extendido es uno de los motivos por los que se utilizan para almacenar datos XML.

○ No.
○ Sí.

2.1.1.- Reglas de transformación relacional. DTD.

La estructura XML debe ser capaz de acomodar algunos conceptos básicos, incluyendo claves primarias, claves secundarias, tablas, y columnas.

El proceso de traducción puede ser descompuesto en los siguientes pasos básicos:

- **Crear el esquema XML** con un elemento para cada tabla y los atributos correspondientes para cada columna no clave. Las columnas que no permiten valores nulos pueden ser marcadas como requeridas, mientras que aquellas que permiten valores nulos pueden ser marcadas como opcionales en el esquema XML. Las columnas pueden ser también anidadas como elementos, pero pueden surgir problemas cuando el mismo nombre de columna es usado en más de una tabla. Por ello, lo más simple es transformar las columnas como atributos XML, donde las colisiones de nombre en el esquema XML no son un problema.
- **Crear las claves primarias en el esquema XML**. Una solución podría ser agregar un atributo para la columna clave primaria, con un ID agregado al nombre de la columna. Este atributo podría necesitar ser definido en el esquema XML como de tipo ID. Pueden surgir problemas de colisión al crear claves primarias en el esquema XML, ya que a diferencia de las bases de datos relacionales, donde las claves primarias necesitan ser únicas sólo dentro de una tabla, un atributo ID dentro de un documento XML debe ser único a través de todo el documento.
  Para resolverlo se puede agregar el nombre del elemento (nombre de la tabla), al valor de la clave primaria (valor del atributo). Esto asegura que el valor es único a través del documento XML.
- **Establecer las relaciones de clave migrada**. Esto se puede lograr mediante el anidamiento de elementos bajo el elemento padre, un ID de esquema XML puede ser usado para apuntar a una estructura XML correspondiente conteniendo un IDREF.

Pueden existir muchas variaciones de esquemas XML para representar la misma base de datos relacional.

Autoevaluación

Relaciona los siguientes pasos con su orden a la hora de transformar una base de datos relacional a almacenamiento XML.

Ejercicio de relacionar

Paso	Relación	Orden
Establecer las relaciones de clave migrada.	☐	1.- Primer lugar.
Crear el esquema XML	☐	2.- Segundo lugar.
Crear las claves primarias	☐	3.- Tercer lugar.

Enviar

2.2.- XML y Bases de Datos Orientadas a Objetos.

Las **bases de datos orientadas a objetos** soportan un modelo de objetos puro, en el sentido de que no están basados en extensiones de otros modelos más clásicos como el relacional:

- Están influenciados por los lenguajes de programación orientados a objetos.
- Pueden verse como un intento de añadir la funcionalidad de un SGBD a un lenguaje de programación.
- Son una alternativa para el almacenamiento y gestión de documentos XML.

Componentes del estándar:

- **Modelo de Objetos.** Está concebido para proporcionar un modelo de objetos estándar para las bases de datos orientadas a objetos. Es el modelo en el que se basan el lenguaje de definición de objetos y el lenguaje de consultas.
- **Lenguajes de Especificación de Objetos.** ODL
- **Lenguaje de Consulta.** Conocido como OQL .
- **'Bindings' para C++, Java y Smalltalk.** Definen un OML que extiende el lenguaje de programación para soportar objetos persistentes. Además incluye soporte para OQL, navegación y transacciones.

Una vez transformado el documento XML en objetos, éstos son gestionados directamente por el SGBDOO. Dicha información se consulta acudiendo al lenguaje de consulta OQL. Los mecanismos de indexación, optimización, procesamiento de consultas, etc. son los del propio SGBDOO, y por lo general, no son específicos para el modelo XML.

Autoevaluación

Selecciona aquellos que sean componentes del estándar de objetos:

○ SQL.
○ OQL.
○ XQuery.
○ XQL.

2.3.- XML y Bases de Datos Nativas.

Son bases de datos, y como tales soportan transacciones, acceso multi-usuario, lenguajes de consulta, etc. Están diseñadas especialmente para almacenar documentos XML.

Las BD nativas se caracterizan principalmente por:

- **Almacenamiento de documentos en colecciones.** Las colecciones juegan en las bases de datos nativas el papel de las tablas en las BD relacionales.
- **Validación de los documentos.**
- **Consultas.** La mayoría de las BD XML soportan uno o más lenguajes de consulta. Uno de los más populares es XQuery.
- **Indexación XML.** Se ha de permitir la creación de índices que aceleren las consultas realizadas.
- **Creación de identificadores únicos.** A cada documento XML se le asocia un identificador único.
- **Actualizaciones y Borrados.**

Según el tipo de almacenamiento utilizado pueden dividirse en dos grupos:

TIPO DE ALMACENAMIENTO	DESCRIPCIÓN
Basado en Texto	Almacena el documento XML entero en forma de texto y proporciona alguna funcionalidad de base de datos para acceder a él. Hay dos posibilidades: • **Posibilidad 1**: Almacenar el documento como un BLOB en una base de datos relacional, mediante un fichero, y proporcionar algunos índices sobre el documento que aceleren el acceso a la información. • **Posibilidad 2**: Almacenar el documento en un almacén adecuado con índices, soporte para transacciones, etc.
Basado en el modelo	Almacena un modelo binario del documento (por ejemplo, DOM) en un almacén existente o bien específico. • **Posibilidad 1**: Traducir el DOM a tablas relacionales como Elementos, Atributos, Entidades, etc. • **Posibilidad 2**: Traducir el DOM a objetos en una BDOO. • **Posibilidad 3**: Utilizar un almacén creado especialmente para esta finalidad.

Autoevaluación

Selecciona aquellas características de las bases de datos nativas:

☐ Triggers.
☐ Consultas.
☐ Creación de índices.
☐ Creación de identificadores únicos.

Mostrar Información

3.- XQuery.

XQuery es un lenguaje diseñado para escribir consultas sobre colecciones de datos expresadas en XML. Puede aplicarse tanto a archivos XML, como a bases de datos relacionales con funciones de conversión de registros a XML. Su principal función es extraer información de un conjunto de datos organizados como un árbol de etiquetas XML. En este sentido XQuery es independiente del origen de los datos.

Permite la construcción de expresiones complejas combinando expresiones simples de una manera muy flexible.

De manera general podemos decir que XQuery es a XML lo mismo que SQL es a las bases de datos relacionales. Al igual que éste último, XQuery es un lenguaje funcional.

Los **requerimientos técnicos** más importantes **de XQuery** se detallan a continuación:

- Debe ser un **lenguaje declarativo**.
- Debe ser **independiente del protocolo de acceso** a la colección de datos. Esto significa que una consulta en XQuery, debe funcionar igual al consultar un archivo local, que al consultar un servidor de bases de datos, o que al consultar un archivo XML en un servidor web.
- Las consultas y los resultados deben respetar el **modelo de datos XML**.
- Las consultas y los resultados deben ofrecer **soporte para los namespaces.**
- Debe soportar **XML-Schemas y DTDs** y también debe ser capaz de trabajar sin ellos.
- Ha de ser **independiente de la estructura** del documento, esto es, funcionar sin conocerla.
- Debe soportar **tipos simples**, como enteros y cadenas, y **tipos complejos**, como un nodo compuesto.
- Las consultas deben soportar **cuantificadores universales** (para todo) y existenciales (existe).
- Las consultas deben soportar **operaciones sobre jerarquías de nodos** y secuencias de nodos.
- Debe ser posible **combinar información de múltiples fuentes** en una consulta.
- Las consultas deben ser capaces de **manipular los datos independientemente del origen** de estos.
- **El lenguaje de consulta debe ser independiente de la sintaxis**, esto es, pueden existir varias sintaxis distintas para expresar una misma consulta en XQuery.

3.1.- Aplicaciones.

Una vez que hemos visto la definición del lenguaje y sus principales requerimientos, queda pensar, ¿para qué se utiliza?

Sus principales aplicaciones se resumen en tres:

- **Recuperar información a partir de conjuntos de datos XML**.
- **Transformar unas estructuras de datos XML** en otras estructuras que organizan la información de forma diferente.
- **Ofrecer una alternativa a XSLT** para realizar transformaciones de datos en XML a otro tipo de representaciones, como HTML o PDF.

¿Y cuáles son los motores XQuery de código abierto más relevantes y sus características principales?

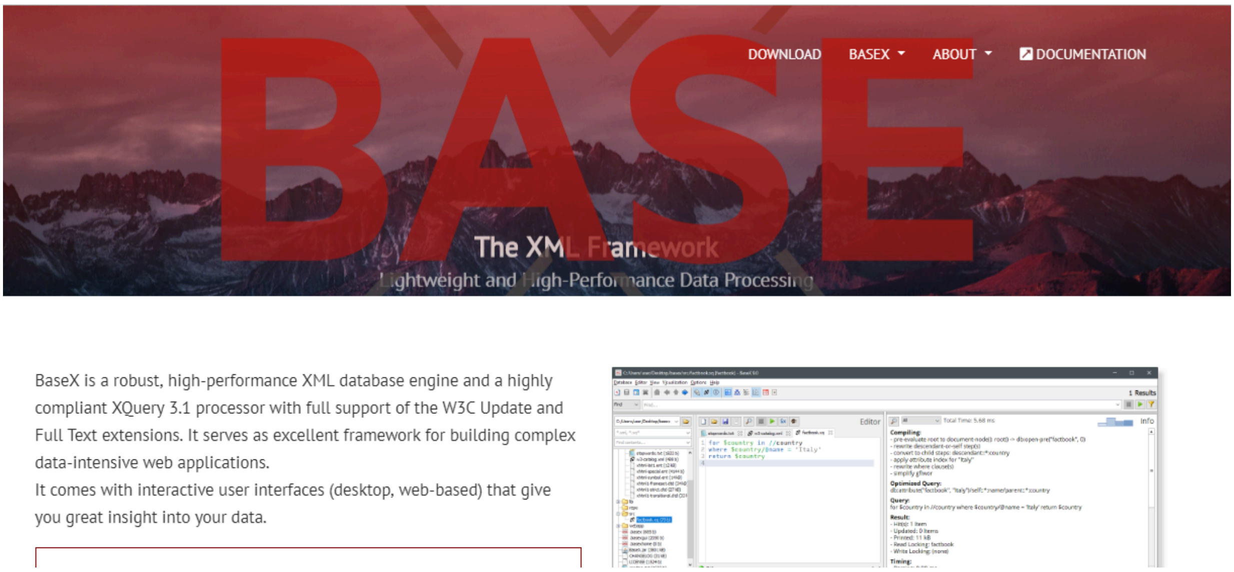
MOTORES XQuery	DESCRIPCIÓN
Qexo	Escrito en Java y con licencia GPL que se distribuye integrado dentro del paquete Kawa.
Saxon	Escrito en Java y distribuido en dos paquetes: • Saxon-B es open-source bajo licencia GPL y contiene una implementación básica de XSLT 2.0 y XQuery. • Saxon-SA, contiene un procesador completo XSLT y XQuery pero tiene licencia propietaria, aunque está disponible una licencia de evaluación de 30 días.
Qizx/open	Es una implementación Java de todas las características del lenguaje excepto la importación y validación de XML-Schemas. Es uno de los motores con licencia GPL más completo que existe.

¿Qué otras herramientas relacionadas con XQuery existen?

HERRAMIENTAS	DESCRIPCIÓN
BaseX	es un sistema de gestión de base de datos XML nativo y liviano y procesador XQuery, desarrollado como un proyecto de comunidad en GitHub. Está especializado en almacenar, consultar y visualizar grandes documentos y colecciones XML. (+ info: http://basex.org/)

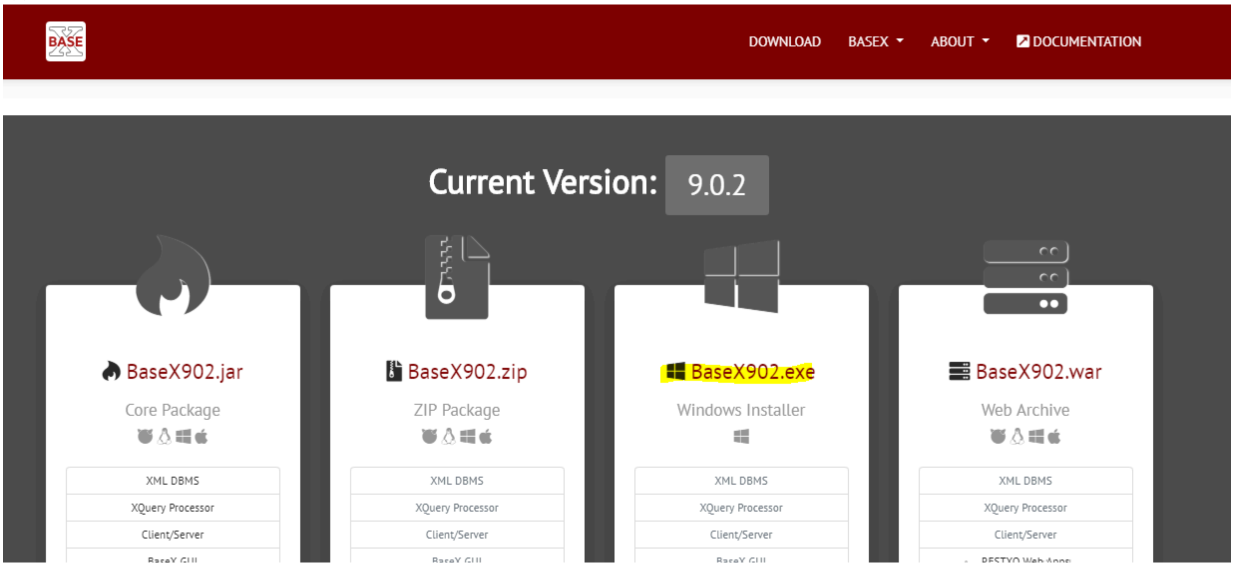
Instalación de Base X

1. En las siguientes capturas se indica cómo instalar Base X que nos servirá para realizar algunas de las consultas XQuery más adelante. Accede a basex.org

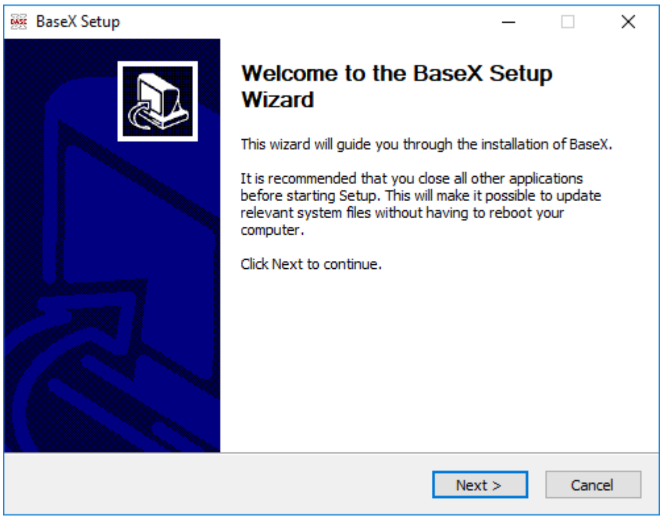


BaseX is a robust, high-performance XML database engine and a highly compliant XQuery 3.1 processor with full support of the W3C Update and Full Text extensions. It serves as excellent framework for building complex data-intensive web applications.
It comes with interactive user interfaces (desktop, web-based) that give you great insight into your data.

1. Seleccionamos la versión para Windows (puedes escoger la de tu sistema operativo).



1. Una vez descargado inicia la descarga, acepta las condiciones de uso, escoge ubicación.

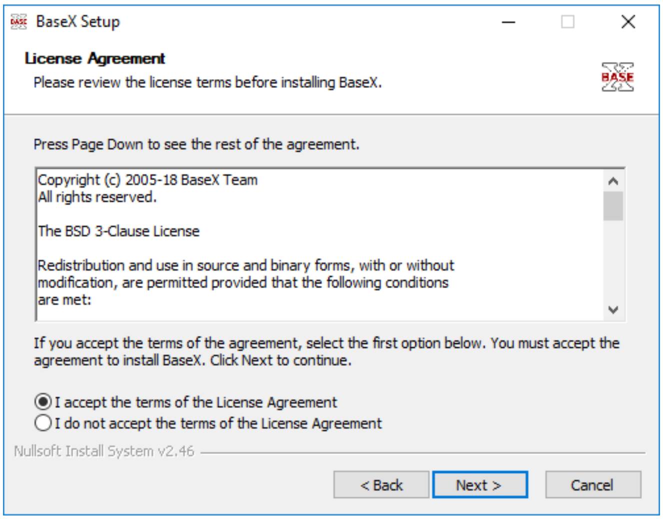


Welcome to the BaseX Setup Wizard

This wizard will guide you through the installation of BaseX.

It is recommended that you close all other applications before starting Setup. This will make it possible to update relevant system files without having to reboot your computer.

Click Next to continue.



License Agreement

Please review the license terms before installing BaseX.

Press Page Down to see the rest of the agreement.

Copyright (c) 2005-18 BaseX Team
All rights reserved.

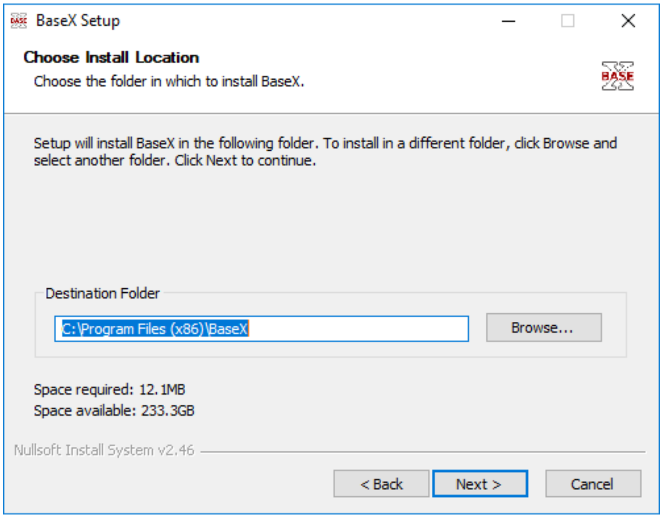
The BSD 3-Clause License

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

If you accept the terms of the agreement, select the first option below. You must accept the agreement to install BaseX. Click Next to continue.

● I accept the terms of the License Agreement
○ I do not accept the terms of the License Agreement

Nullsoft Install System v2.46



Choose Install Location

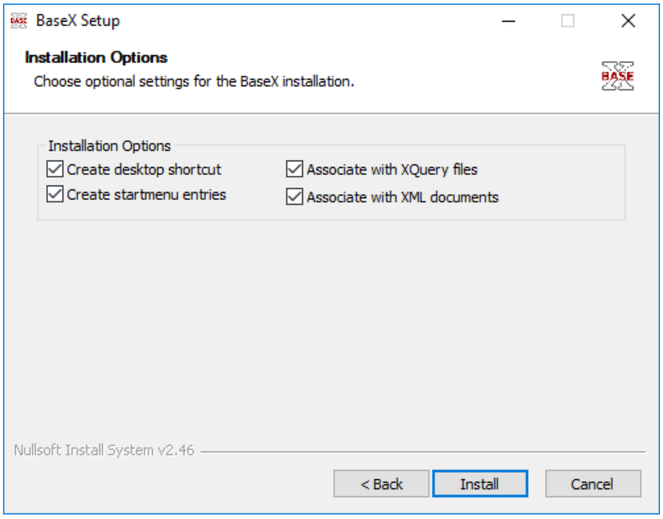
Choose the folder in which to install BaseX.

Setup will install BaseX in the following folder. To install in a different folder, click Browse and select another folder. Click Next to continue.

Destination Folder: C:\Program Files (x86)\BaseX

Space required: 12.1MB
Space available: 233.3GB

Nullsoft Install System v2.46

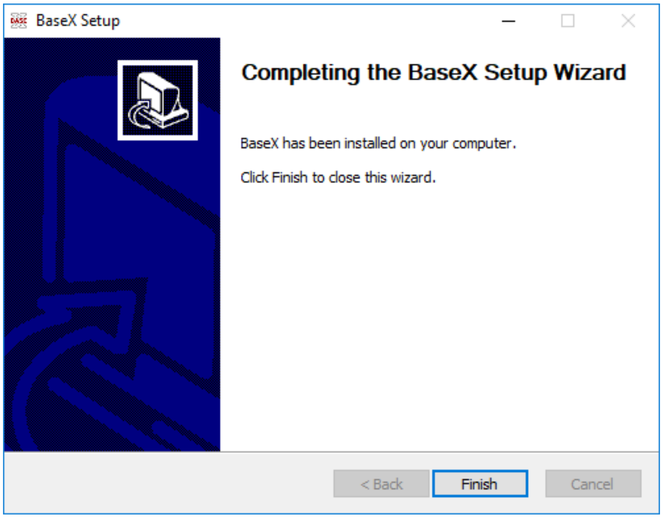


Installation Options

Choose optional settings for the BaseX installation.

Installation Options
☑ Create desktop shortcut
☑ Create startmenu entries
☑ Associate with XQuery files
☑ Associate with XML documents

Nullsoft Install System v2.46

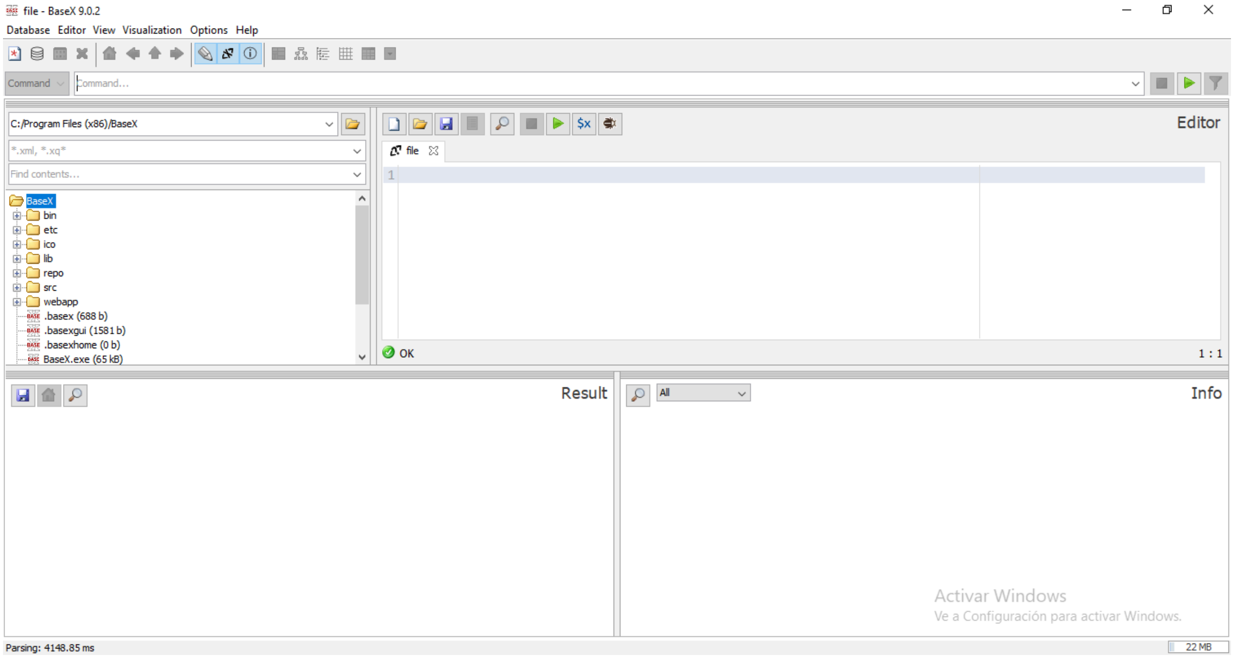


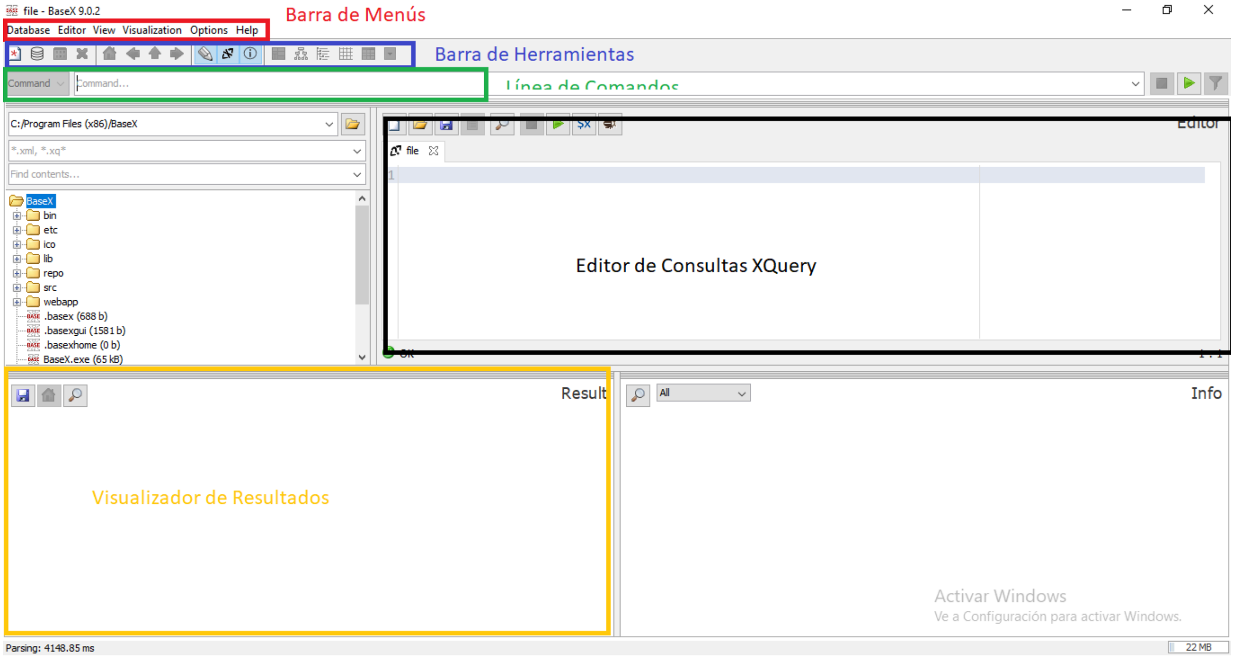
Completing the BaseX Setup Wizard

BaseX has been installed on your computer.

Click Finish to close this wizard.

1. Aquí tendremos la página prinicipal de la aplicación Base-X


file - BaseX 9.0.2
Database Editor View Visualization Options Help
Command
Command...
C:/Program Files (x86)/BaseX
*.xml, *.xq*
Find contents...
BaseX
bin
etc
ico
lib
repo
src
webapp
.basex (688 b)
.basexgui (1581 b)
.basexhome (0 b)
BaseX.exe (65 kB)
Editor
file
OK
1 : 1
Result
All
Info
Parsing: 4148.85 ms
22 MB


Barra de Menús
Barra de Herramientas
Línea de Comandos
Editor de Consultas XQuery
Visualizador de Resultados
file - BaseX 9.0.2
Database Editor View Visualization Options Help
Command
Command...
C:/Program Files (x86)/BaseX
Editor
Result
Info
Parsing: 4148.85 ms
22 MB

3.2.- Modelo de datos.

Aunque XQuery y SQL puedan considerarse similares, el **modelo de datos** sobre el que se sustenta XQuery es muy distinto del modelo de datos relacional sobre el que sustenta SQL, ya que XML incluye conceptos como jerarquía y orden de los datos que no están presentes en el modelo relacional.

Por ejemplo, a diferencia de SQL, en XQuery el orden en que se encuentren los datos es importante, ya que no es lo mismo buscar una etiqueta “” **dentro de una etiqueta** “” que todas las etiquetas “” del documento (que pueden estar anidadas dentro de una etiqueta “” o no).

La entrada y la salida de una consulta XQuery se define en términos de un modelo de datos. Dicho modelo de datos de la consulta proporciona una representación abstracta de uno o más documentos XML (o fragmentos de documentos).

Las principales características de este modelo de datos son:

- Se basa en la **definición de secuencia**, como una colección ordenada de cero o más ítems. Éstas pueden ser heterogéneas, es decir pueden contener varios tipos de nodos y valores atómicos. Sin embargo, una secuencia nunca puede ser un ítem de otra secuencia.
- **Orden del documento:** corresponde al orden en que los nodos aparecerían si la jerarquía de nodos fuese representada en formato XML, (si el primer carácter de un nodo ocurre antes que el primer carácter de otro nodo, lo precederá también en el orden del documento).
- Contempla un **valor especial llamado “error value”** que es el resultado de evaluar una expresión que contiene un error.

Autoevaluación

Alguna de las características del modelo de datos en el que se basa XQuery es:

○ El orden de los nodos no importa.
○ Se basa en secuencias de ítems, donde las secuencias se anidan dentro de otras secuencias.
○ La secuencia es un conjunto de datos de cualquier tipo.
○ Tiene un valor especial que aparece cuando se trata de evaluar una expresión errónea.

Debes conocer

En el siguiente enlace puedes encontrar el estándar de XQuery aprobado por el W3C en diciembre de 2010.

Recomendación del W3C sobre XQuery versión 1.0.

3.3.- Expresiones.

Una consulta en XQuery es una expresión que lee una secuencia de datos en XML, y devuelve como resultado otra secuencia de datos en XML.

El valor de una expresión es una secuencia heterogénea de nodos y valores. La mayoría de las expresiones están compuestas por la combinación de expresiones más simples unidas mediante operadores y palabras reservadas.

Ya hemos visto que Xpath es un lenguaje declarativo para la localización de nodos y fragmentos de información en árboles XML. Puesto que XQuery ha sido construido sobre la base de Xpath y realiza la selección de información y la iteración a través del conjunto de datos basándose en dicho lenguaje, toda expresión XPath también es una consulta Xquery válida.

Los comentarios en XQuery están limitados entre caras sonrientes, es decir:

(: Esto es un comentario XQuery :).

En un documento XQuery los caracteres { } delimitan las expresiones que son evaluadas para crear un documento nuevo.

XQuery admite expresiones condicionales del tipo if-then-else con la misma semántica que tienen en los lenguajes de programación habituales.

Las consultas XQuery pueden estar formadas por hasta cinco tipos de cláusulas diferentes, siguen la norma FLWOR (que se pronuncia "flower" y proviene de las siglas de For, Let, Where, Order by y Return). Estas cláusulas son los bloques principales del XQuery, equivalen a las cláusulas select, from, where, group by, having, order by y limit de SQL.

En una sentencia FLWOR al menos ha de existir una cláusula FOR o una LET, el resto, si existen, han de respetar escrupulosamente el orden dado por el nombre, FLWOR.

Con estas sentencias se consigue buena parte de la funcionalidad que diferencia a XQuery de XPath. Entre otras cosas permite construir el documento que será la salida de la sentencia.

Una consulta XQuery está formada por dos partes:

- **Prólogo: Lugar donde se declaran los espacios de nombres, de funciones, variables, etc.**
- **Expresión: Consulta propiamente dicha.**

Vamos a explicar un primer ejemplo extraído del tutorial del W3CSchool sobre XQuery

Debemos conocer que la función doc("fichero.xml") sirve para abrir el fichero XML sobre el cual vamos a realizar la consulta y extraer la información.

Partiendo del documento XML bookstore.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>

<book category="COOKING">
 <title lang="en">Everyday Italian</title>
 <author>Giada De Laurentiis</author>
 <year>2005</year>
 <price>30.00</price>
</book>

<book category="CHILDREN">
 <title lang="en">Harry Potter</title>
 <author>J K. Rowling</author>
 <year>2005</year>
 <price>29.99</price>
</book>

<book category="WEB">
 <title lang="en">XQuery Kick Start</title>
 <author>James McGovern</author>
 <author>Per Bothner</author>
 <author>Kurt Cagle</author>
 <author>James Linn</author>
 <author>Vaidyanathan Nagarajan</author>
 <year>2003</year>
 <price>49.99</price>
</book>

<book category="WEB">
 <title lang="en">Learning XML</title>
 <author>Erik T. Ray</author>
 <year>2003</year>
 <price>39.95</price>
</book>

</bookstore>
```

Ejemplo 1: Extraer todos los títulos

La consulta XQuery: doc("books.xml")/bookstore/book/title

obtiene:

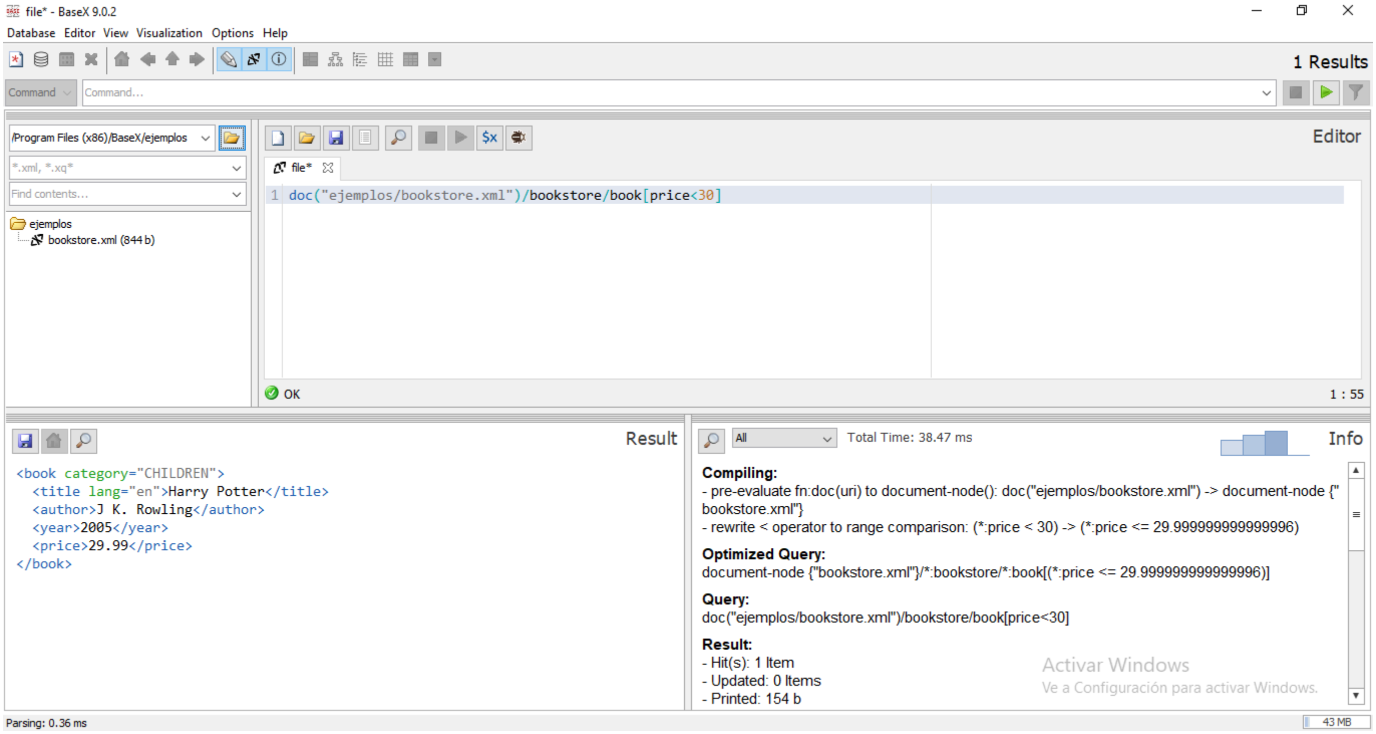
```
<title·lang="en">Everyday·Italian</title>
<title·lang="en">Harry·Potter</title>
<title·lang="en">XQuery·Kick·Start</title>
<title·lang="en">Learning·XML</title>
```

Ejemplo 2: Seleccionar los libros cuyo precio sea menor de 30 dólares La consulta XQuery: **doc("books.xml")/bookstore/book[price<30]**

obtiene:

```
<book·category="CHILDREN">
  <title·lang="en">Harry·Potter</title>
  <author>J·K.·Rowling</author>
  <year>2005</year>
  <price>29.99</price>
</book>
```

Ahora prueba a ejecutarlo con Base-X. Ubica el archivo bookstore.xml dentro de la carpeta BaseX donde instalaste la aplicación y copia alguna de las dos expresiones de ejemplo en el editor de consultas XQuery... tienes que obtener un resultado similar al que se muestra en la captura.



```
doc("ejemplos/bookstore.xml")/bookstore/book[price<30]
```

```
<book category="CHILDREN">
  <title lang="en">Harry Potter</title>
  <author>J K. Rowling</author>
  <year>2005</year>
  <price>29.99</price>
</book>
```

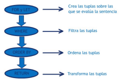
```
Total Time: 38.47 ms

Compiling:
- pre-evaluate fn:doc(uri) to document-node(): doc("ejemplos/bookstore.xml") -> document-node {"
bookstore.xml"}
- rewrite < operator to range comparison: (*:price < 30) -> (*:price <= 29.999999999999996)

Optimized Query:
document-node {"bookstore.xml"}/*:bookstore/*:book[(*:price <= 29.999999999999996)]

Query:
doc("ejemplos/bookstore.xml")/bookstore/book[price<30]

Result:
- Hit(s): 1 Item
- Updated: 0 Items
- Printed: 154 b

Parsing: 0.36 ms
```

Autoevaluación

Toda sentencia XPath es una sentencia XQuery:

○ **Verdadero.**
○ **Falso.**

Ejecución de una consulta FLWOR:



Resumen textual alternativo

3.4.- Cláusulas.

Hemos visto el modo de crear sentencias FLWOR, vamos ahora a estudiar aisladamente cada una de las cláusulas que pueden formar estas sentencias.

CLÁUSULAS	DESCRIPCIÓN
FOR	Asocia una o más variables con cada nodo que encuentre en la colección de datos. Si en la consulta aparece más de una cláusula FOR (o más de una variable en una cláusula FOR), el resultado es el producto cartesiano de dichas variables.
LET	Vincula las variables al resultado de una expresión. Si esta cláusula aparece en una sentencia en la que ya hay al menos una cláusula FOR, los valores de la variable vinculada por la cláusula LET se añaden a cada una de las tuplas generadas por la cláusula FOR.
WHERE	Filtra tuplas producidas por las cláusulas FOR y LET, quedando solo aquellas que cumplen con la condición.
ORDER BY	Ordena las tuplas generadas por FOR y LET después de que han sido filtradas por la cláusula WHERE. Por defecto el orden es ascendiente, pero se puede usar el modificador **descending** para cambiar el sentido del orden.
RETURN	Construye el resultado de la expresión FLWOR para una tupla dada.

Recuerda que puedes practicar todos estos ejemplos en la aplicación Base-X.

Ejemplo:

Partiendo del documento bookstore.xml del apartado anterior, vamos a analizar la siguiente expresión FLWOR:

```
for $x in doc("books.xml")/bookstore/book

where $x/price>30

return $x/title
```

La cláusula for selecciona todos los elementos book que hay dentro del elemento raíz bookstore dentro de una variable llamada $x. La cláusula where selecciona solamente los elementos book con un elemento price con un valor mayor de 30. La cláusula return especifica qué debe ser devuelto, en este caso, los elementos title.El resultado de la consulta XQuery será:

```
<title lang="en">XQuery Kick Start</title>
<title lang="en">Learning XML</title>
```

Si a la anterior expresión FLWOR le añadiésemos una cláusula ORDER, el resultado sería ordenado alfabéticamente:

```
for $x in doc("books.xml")/bookstore/book
where $x/price>30
order by $x/title
return $x/title
<title lang="en">Learning XML</title>

<title lang="en">XQuery Kick Start</title>
```

Para saber más

En este post puedes ver más ejemplos y consultas XQuery con expresiones FLWOR más elaboradas.

Ejercicio resuelto

Veamos unos ejemplos de la forma de uso de estas cláusulas. Para ello partiremos del fichero XML de datos:

```
Learning XML

Ray
Erik T.

O'Reilly

616


XML Imprescindible

Harold
Elliot Rusty


Means
W. Scott

O'Reilly

832


XML Schema

van der Vlist
Eric

O'Reilly

600


XPath Essentials

Watt
Adrew

Wiley

516


 Beginning XSLT 2.0: Form Novice to Professional

Tennison
Jeni

Apress

797


 XQuery

Walmsley
Priscilla

O'Reilly

491




XML Imprescindible

2011-05-02

Pepito
Grillo
Rue Percebe, 13
972987656



XML Imprescindible

2011-02-12
2011-02-16

Jose
Gutiérrez González
Rue Percebe, 13
919485632



XPath Essentials

2011-02-23
2011-03-10

Pepito
Grillo
Rue Percebe, 13
972987656
```

1. **Ejemplo de uso de la cláusula FOR. Obtener todos los títulos de los libros del fichero libros.xml.**
2. **Ejemplo de uso de la cláusula LET. Obtener todos los títulos de los libros del fichero libros.xml.**
3. **Ejemplo de uso de la cláusula FOR y LET juntas. Obtener todos los títulos de los libros del fichero libros.xml junto con los autores de cada libro.**
4. **Ejemplo de uso de las cláusulas WHERE y ORDER BY en una consulta con dos ficheros. Obtiene los títulos de los libros prestados con sus autores y la fecha de inicio y devolución del préstamo, ordenados por la fecha de inicio del préstamo.**

3.5.- Funciones.

Ahora que conocemos las cláusulas que sustentan el lenguaje XQuery, vamos a ocuparnos de las funciones que soporta.
Estas son funciones matemáticas, de cadenas, para el tratamiento de expresiones regulares, comparaciones de fechas y horas, manipulación de nodos XML, manipulación de secuencias, comprobación y conversión de tipos y lógica booleana. Además permite definir funciones propias y funciones dependientes del entorno de ejecución del motor XQuery. Las funciones más importantes se muestran a continuación:

TIPOS	FUNCIONES
Funciones numéricas	• **floor()**, que devuelve el valor numérico inferior más próximo al dado. • **ceiling()**, que devuelve el valor numérico superior más próximo al dado. • **round()**, que redondea el valor dado al más próximo. • **count()**, determina el número de ítems en una colección. • **min()** o **max()**, devuelven respectivamente el mínimo y el máximo de los valores de los nodos dados. • **avg()**, calcula el valor medio de los valores dados. • **sum()**, calcula la suma total de una cantidad de ítems dados.
Funciones de cadenas de texto	• **concat()**, devuelve una cadena construida por la unión de dos cadenas dadas. • **string-length()**, devuelve la cantidad de caracteres que forman una cadena. • **startswith(), ends-with()**, determinan si una cadena dada comienza o termina, respectivamente, con otra cadena dada. • **upper-case(), lower-case()**, devuelve la cadena dada en mayúsculas o minúsculas respectivamente.
Funciones de uso general	• **empty()**, devuelve "**true**" cuando la secuencia dada no contiene ningún elemento. • **exits()**, devuelve "**true**" cuando una secuencia contiene, al menos, un elemento. • **distinct-values()**, extrae los valores de una secuencia de nodos y crea una nueva secuencia con valores únicos, eliminando los nodos duplicados.
Cuantificadores existenciales	• **some, every**, permiten definir consultas que devuelven algún, o todos los elementos, que verifiquen la condición dada.

> Ejercicio resuelto
>
> **Dados los ficheros XML del ejemplo anterior, hacer una consulta que devuelva los títulos de los libros almacenados en el fichero libros.xml y su primer autor. En caso de que haya más de un autor para un libro se añade un segundo autor "cia".**

Además de estas funciones que están definidas en el lenguaje XQuery permite al desarrollador construir sus propias funciones:

```
declare nombre_funcion($param1 as tipo_dato1, $param2 as tipo_dato2,... $paramN as tipo_datoN)
as tipo_dato_devuelto
{
...CÓDIGO DE LA FUNCIÓN...
}
```

> Ejercicio resuelto
>
> **Ejemplo de la creación y uso de funciones de usuario:**

3.6.- Operadores.

Veamos ahora algunos de los operadores más importantes agrupados según su funcionalidad:

OPERADORES	FUNCIONALIDAD
Comparación de valores	Comparan dos valores escalares y produce un error si alguno de los operandos es una secuencia de longitud mayor de 1. Estos operadores son: • **eq**, igual. • **ne**, no igual. • **lt**, menor que. • **le**, menor o igual que. • **gt**, mayor que. • **ge**, mayor o igual que.
Comparación generales	Permiten comparar operandos que sean secuencias. • **=**, igual. • **!=**, distinto. • **>**, mayor que. • **>=**, mayor o igual que. • **<**, menor que. • **<=**, menor o igual que.
Comparación de nodos	Comparan la identidad de dos nodos. • **is**, devuelve true si las dos variables que actúan de operandos están ligadas al mismo nodo. • **is not**, devuelve true si las dos variables no están ligadas al mismo nodo.
Comparación de órdenes de los nodos	• **<<**, compara la posición de dos nodos. Devuelve • "**true**" si el nodo ligado al primer operando ocurre primero en el orden del documento que el nodo ligado al segundo.
Lógicos	• and y or: Se emplean para combinar condiciones lógicas dentro de un predicado.
Secuencias de nodos	Devuelven secuencias de nodos en el orden del documento y eliminan duplicados de las secuencias resultado. • **Unión**, devuelve una secuencia que contiene todos los nodos que aparecen en alguno de los dos operandos que recibe. • **Intersect**, devuelve una secuencia que contiene todos los nodos que aparecen en los dos operandos que recibe. • **Except**, devuelve una secuencia que contiene todos los nodos que aparecen en el primer operando que recibe y que no aparecen en el segundo.
Aritméticos: +, -, *, div y mod	• +, -, *, div y mod: devuelven respectivamente la suma, diferencia, producto, cociente y resto de operar dos números dados.

Autoevaluación

Cuáles de los siguientes operadores pertenecen a SQL y a XQuery:

☐ **is.**
☐ **union.**
☐ **lt.**
☐ **>=.**

Mostrar Información

Anexo.- Licencias de recursos.

Licencias de recursos utilizados en la Unidad de Trabajo.

Recurso (1)	Datos del recurso (1)	Recurso (2)	Datos del recurso (2)
	Autoría: nate steiner. Licencia: CC By. Procedencia: http://www.flickr.com/photos/nate/2754438913/in/photostream/		Autoría: nate steiner. Licencia: CC By. Procedencia: http://www.flickr.com/photos/nate/2575640907/in/photostream/
	Autoría: xadrian. Licencia: CC By-nc-sa. Procedencia: http://www.flickr.com/photos/xadrian/4574250983/		Autoría: Foreverdigital. Licencia: CC by-nc-nd. Procedencia: http://www.flickr.com/photos/foreverdigital/357103120
	Autoría: Let Ideas Compete. Licencia: CC By-nc-nd. Procedencia: http://www.flickr.com/photos/question_everything/4077360283/in/set-72157606482779336/		Autoría: Jorge glez. Licencia: CC By-nc. Procedencia: http://www.flickr.com/photos/glztt/502363070/
	Autoría: -ELFiagon-. Licencia: CC By-nc-nd. Procedencia: http://www.flickr.com/photos/salya_yasal/2942806332/		Autoría: Leo Reynolds. Licencia: CC By-nc-sa. Procedencia: http://www.flickr.com/photos/lwr/517732451/
	Autoría: reiven. Licencia: CC By-nc-sa. Procedencia: http://www.flickr.com/photos/reiven/3060319319/		Autoría: adesigna. Licencia: CC by-nc-sa. Procedencia: http://www.flickr.com/photos/adesigna/4967712037
	Autoría: jonny goldstein. Licencia: CC By. Procedencia: http://www.flickr.com/photos/jonnygoldstein/2311081494/		Autoría: Anne Helmond. Licencia: CC By-nc-nd. Procedencia: http://www.flickr.com/photos/silvertje/4100459062/
	Autoría: Gory. Licencia: Ccy By-nc-sa. Procedencia: http://www.flickr.com/photos/lordgoroth/2024226436/		Autoría: happyclair. Licencia: CC By-sa. Procedencia: http://www.flickr.com/photos/eclair/849685926/
	Autoría: seferman. Licencia: CC By-nc-sa. Procedencia: http://www.flickr.com/photos/seferman/2601485302/		Autoría: Galería de JoshuaDavisPhotography. Licencia: CC by-sa. Procedencia: http://www.flickr.com/photos/ranzio/2829083155
	Autoría: mollyali. Licencia: CC By-nc. Procedencia: http://www.flickr.com/photos/mollyali/2035234593/		Autoría: Daryl_mitchell. Licencia: CC by-nc-sa. Procedencia: http://www.flickr.com/photos/daryl_mitchell/1199598508